Results and Analysis

I benchmarked the app_helper system call across four buffer sizes (64B, 256B, 4KB, 64KB) with 10 trials each. A warmup phase of 1,000 untimed iterations was run before each benchmark to warm up caches and reduce cold-start effects like TLB misses and page faults, as discussed in class. Timing was done using clock_gettime(CLOCK_MONOTONIC) for monotonic measurements. I also only set memset once before starting the iterations so that the latency of memset doesn't affect the actual benchmark.

For a 64-byte buffer (200,000 iterations per trial), Trial 1 averaged 1,308.23 ns due to cold-start effects, but latency stabilized around Trial 5 to approximately 742-755 ns (around 0.75 us). The 256-byte buffer (200,000 iterations per trial) showed very similar behavior, with Trial 1 at 1,257.97 ns and steady-state trials settling to around 752-764 ns (around 0.76 us). The near-identical latencies between these two sizes show that the fixed overhead of the user-kernel space transition dominates when the data being copied is small

For the 4,096-byte buffer (100,000 iterations per trial), Trial 1 started at 1,344.22 ns and the later trials stabilized around 940-970 ns (around 0.95 us), showing a increase from the smaller buffers.
At 65,536 bytes (20,000 iterations per trial), latency increased significantly. Trial 1 averaged 14,983.12 ns, while steady-state trials ranged from 12,722-12,882 ns (around 12.8 us). A second run at this same buffer size produced very consistent results with steady-state latencies of 12,695-13,136 ns (around 12.8 us), confirming good reproducibility.

Across all buffer sizes, Trial 1 was consistently the slowest due to residual cache warmup effects beyond the warmup phase. Latencies stabilized by around Trial 5-6 and showed low variability afterward, with steady-state measurements varying by only ~10-20 ns for small buffers and ~150 ns for the 64KB buffer.